

Base de données NoSQL

Réplication

Louis Roussel

louis.roussel@univ-lille.fr

Support de cours par **Bastien Cazaux** qui s'appuie sur les cours d'**Anne Etien** et de **Anne Cécile Caron** de l'université de Lille et celui de **Régis Behmo** et **Nicolas Travers** de CentralSupélec.

Plan du cours

Plan du cours		
Introduction	3	
Fragmentation et réplication		
Réplication : avantages et inconvénients		
Gestion des réplicas : cohérence vs latence	5	
Réplication		
Réplication primaire – secondaire		
Réplication primaire – secondaire : Forces		
et Faiblesses		
Réplication <i>peer-to-peer</i>		
Combiner fragmentation et réplication		
Mise à jour : gérer la concurrence		
Gestion de la consistance		
Relâcher la consistance		
Théorème CAP		13
Le théorème CAP (Eric A. Brewer 2000)		
Triangle CAP		

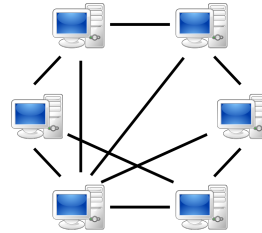
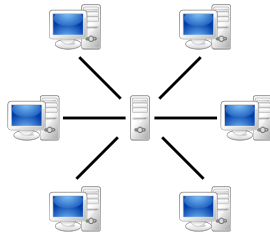
Fragmentation et réplication

- **Fragmentation** : action de partitionner l'ensemble des données en sous-ensembles (fragments) afin de les distribuer sur différentes machines.
- **Réplication** : action de créer des copies des données afin de les distribuer sur différentes machines. Une copie est appelée réplique.
- En général, les deux sont combinés : chaque fragment est répliqué sur plusieurs machines.

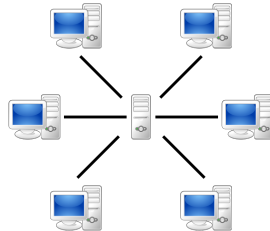
Réplication : avantages et inconvénients

- La réplication permet de :
 - distribuer les lectures/écritures unitaires (extensibilité)
 - avoir une copie des données en cas de perte d'un disque (tolérance aux pannes)
- En contrepartie, elle a aussi des inconvénients :
 - Écrire une donnée avec plusieurs copies va prendre plus de temps qu'une écriture simple (performances)
 - La réplication complique la gestion des transactions (cohérence). Il faut s'assurer de la synchronisation des données
- Nous allons voir plusieurs manières de gérer la réplication, avec le compromis à trouver entre cohérence et latence.
 - la réplication peut être :
 - **synchrone** : quand une donnée est écrite, elle est immédiatement répliquée
 - **asynchrone** : quand une données est écrite, elle va être recopiée plus tard sur les répliques
 - les répliques peuvent être organisés :
 - **hiérarchiquement** : il y a un réplique primaire et des répliques secondaires
 - **indifféremment** : tous les répliques ont le même rôle

- Données répliquées sur plusieurs noeuds
- Deux grands modèles de distribution :
 - Primaire/Secondaire :
 - *Peer-to-peer* : possibilité d'écrire sur chaque



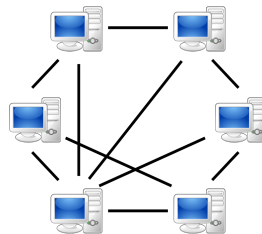
Réplication primaire – secondaire



- Fonctionnement :
 - Un noeud est désigné comme primaire
 - Le noeud primaire est la source de donnée qui fait autorité (responsable des mises à jours et écritures)
 - Les autres noeuds sont des noeuds secondaires (lectures)
- adapté aux cas de lecture intensives
- Possibilité de gérer plus de requêtes de lecture en ajoutant des noeuds secondaires
- Les mises à jours sont gérées exclusivement par le noeud primaire
 - Limitation à ses capacités à gérer les mises à jours et les propager aux noeuds secondaires

- Forces
 - Résilience en écriture
 - Même si le noeud primaire tombe, les noeuds secondaires peuvent continuer les requêtes en lecture
 - La gestion des écritures se fera dès que le noeud primaire sera restauré ou un nouveau serveur assigné en tant que noeud primaire
 - Qualité intéressante même sans problème de passage à l'échelle
- Faiblesses
 - Inconsistance des données
 - Différents clients lisant sur des serveurs différents peuvent avoir des valeurs différentes
 - La propagation des changements n'ayant pas eu lieu sur tous les noeuds secondaires
 - Un client peut éventuellement ne pas lire la mise à jour qu'il vient de faire
 - Noeud primaire = goulot d'étranglement pour écriture

Réplication *peer-to-peer*



- Fonctionnement :
 - Pas de noeud primaire : chaque réplique a le même poids
 - Tous peuvent faire des écritures
- Forces :
 - La perte de l'un d'entre eux n'empêche pas l'accès à des données
 - Possibilité d'augmenter le nombre de noeuds pour augmenter la performance
- Faiblesses :
 - Consistance
 - Ecriture de plusieurs endroits : tentative de mettre à jour la même donnée
 - Inconsistance en écriture
- Solutions :
 - Communication aux autres noeuds après chaque écriture
 - ou accepter l'inconsistance en écriture

- Plusieurs noeuds primaires
- mais chaque donnée n'a qu'un seul noeud primaire
- Un noeud peut être primaire pour certaines données et secondaires pour d'autres
- ou uniquement primaire ou secondaire
- Un bon point de départ est que chaque fragment soit présent sur trois noeuds

Mise à jour : gérer la concurrence

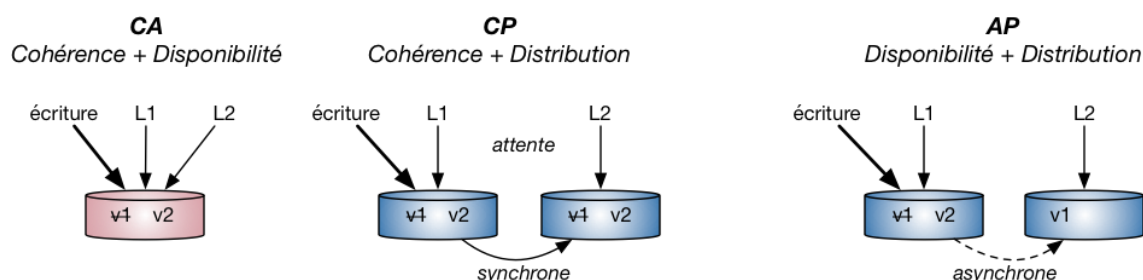
- Si un seul serveur :
 - On choisit une MAJ puis l'autre
- Si plusieurs serveurs :
 - L'ordre peut être différent et la valeur différente sur chacun des serveurs.
 - Sequential consistency : les changements sont faits dans le même ordre pour chacun des serveurs
- Problème de deadlock pour l'approche pessimiste
- NoSQL peut supporter les transactions
 - Les BD graphes supportent les transactions ACID
 - Les BD orientées agrégats supportent les maj atomiques MAIS au sein d'un même agrégat
- Impossible d'avoir un seul agrégat
 - Fenêtre d'inconsistance (moins de 2s pour le SimpleDB service d'Amazon)

- Fenêtre d'inconsistance : des personnes différentes verront au même instant des choses différentes
 - particulièrement problématique quand inconsistance avec soi-même
 - Ex : Poster un message sur un blog mais load-balancing de différents noeuds donc impression d'avoir perdu le post.
 - Besoin de read-your-writes consistance via des sessions par exemple
 - Garantir la consistance ne se fait pas globalement au sein de l'application.
 - Possibilité de spécifier la niveau de consistance pour chaque requête
 - Faible consistance la majeure partie du temps et forte consistance quand nécessaire

Relâcher la consistance

- La consistance est une bonne chose MAIS malheureusement il faut parfois la sacrifier
 - Possible de concevoir un système évitant l'inconsistance, mais souvent au détriment d'autres qualités du système
 - Compromis entre la consistance et d'autres qualités
- Différents domaines ont des tolérances différentes à l'inconsistance.
 - Cette tolérance doit être prise en compte lors du design
- Même problématique en relationnel
 - Doit-on faire des transactions ? A quel niveau d'isolation ?

- Souvent perçu comme : Entre Consistance (Consistency), Disponibilité (Availability) et Distribution (Partition Tolerance), on ne peut en avoir que deux.
- Disponibilité (Availability) : si on peut parler à un noeud dans le cluster, il peut lire et écrire les données
- Distribution (Partition Tolerance) : le cluster peut survivre à la rupture d'une communication dans le cluster qui sépare le cluster en plusieurs partitions incapables de communiquer



Triangle CAP

